

智慧的策略

贪心、并查集与数学 · 8题 · C++ & Python 双语对照

NQ154 **AcWing 905** AcWing 905

AcWing 905

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

struct Range
{
    int l, r;
    bool operator< (const Range & R) const
    {
        return r < R.r; //重载比较符, 根据r排序
    }
} range[N];

int main()
{
    int n;
    scanf("%d", &n);
    //读入区间
    for (int i = 0; i < n; i++)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        range[i] = {l, r};
    }
    //按照右端点排序
    sort(range, range+n);
    //根据右端点计算覆盖的区间个数
    int res = 0, ed = -2e9; //已统计区间的右端点
    for (int i = 0; i < n; i++)
    {
        if (range[i].l > ed)
        {
            res++;
            ed = range[i].r;
        }
    }
    printf("%d\n", res);

    return 0;
}

```

Python

AcWing 905

NQ155 AcWing 148 AcWing 148

AcWing 148

输入

见原题

输出

见原题

范围

见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
#include <iostream>
#include <queue>

using namespace std;

int main()
{
    int n; cin>>n;
    //优先队列, 使用greater作比较函数, 把默认的大根堆
    变为小根堆
    priority_queue<int, vector<int>, greater<int>
>> heap;
    //把读入数据压入堆
    while (n--)
    {
        int x; cin>>x;
        heap.push(x);
    }

    int res = 0;
    while(heap.size() > 1)
    {
        int x = heap.top(); heap.pop();
        int y = heap.top(); heap.pop();
        res += x + y; //累计合并值
        heap.push(x+y); //把当前合并的值压入堆
    }

    cout<<res<<endl;

    return 0;
}
```

Python

AcWing 148

NQ156 AcWing 836 AcWing 836

AcWing 836

输入 见原题

输出 见原题

范围 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
using namespace std;
const int N = 100007;
int p[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}
void merge(int a, int b)
{
    p[find(a)] = find(b);
}
void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}
int main()
{
    int m;
    cin>>n>>m; //输入n
    //创造并查集数
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m--)
    {
        char op;
        int a, b;
        cin>>op;
        cin>>a>>b;
        if (op == 'M') merge(a, b);
        else if (op == 'Q') query(a, b);
    }
    return 0;
}

```

Python

AcWing 836

NQ157 AcWing 837 AcWing 837

AcWing 837

输入 见原题

输出 见原题

范围 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cstring>

using namespace std;
const int N = 100007;
int p[N], Size[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

void merge(int a, int b)
{
    Size[find(b)] += Size[find(a)];
    p[find(a)] = find(b);
}

void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}

int count(int x)
{
    return Size[find(x)];
}

int main()
{
    int m;
    cin >> n >> m; //输入n
    //创建并查集数
    for (int i = 1; i <= n; i++) p[i] = i, Size[i] = 1;
    while (m--)
    {
        string op;
        int a, b;
        cin >> op;
        if (op == "C")
        {
            cin >> a >> b;
            if (find(a) == find(b)) continue; //已经在
            同一个集合里, 跳过merge操作
            merge(a, b);
        }
        else if (op == "Q1") cin >> a >> b, query(a, b);
        else cin >> a, cout << count(a) << endl;
    }
    return 0;
}

```

Python

AcWing 837

NQ158 AcWing 240 AcWing 240

AcWing 240

输入 见原题**输出** 见原题**范围** 见原题输入
见原题输出
见原题**思路** 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```

#include <iostream>

using namespace std;

const int N = 50007; // Andy风格

int n, m;
int p[N], d[N]; // p集合 d距离初始化为0

int find(int x)
{
    if (p[x] != x)
    {
        int t = find(p[x]);
        d[x] += d[p[x]];
        p[x] = t;
    }
    return p[x];
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++) p[i] = i; // 初始化p集合

    int res = 0;
    while (m --)
    {
        int t, x, y;
        scanf("%d%d%d", &t, &x, &y);

        if (x > n || y > n) res ++;
        else
        {
            int px = find(x), py = find(y); // 两个集合的父节点
            if (t == 1)
            {
                if (px == py && (d[x] - d[y]) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] - d[x];
                }
            }
            else
            {
                if (px == py && (d[x] - d[y] - 1) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] + 1 - d[x];
                }
            }
        }

        printf("%d\n", res);
    }
}

```

Python

AcWing 240

```
return 0;  
}
```

NQ159 AcWing 875 AcWing 875

AcWing 875

输入 见原题

输出 见原题

范围 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
typedef long long LL;
int qmi(int a, int b, int p)
{
    int res = 1; //初始值为1
    while (b)
    {
        if (b & 1) //b的最末尾为
1
            res = (LL)res * a % p; //必须用long
long防止溢出
        b >>= 1; //去掉b最低位
        a = (LL)a * a % p;
    }
    return res;
}

int main()
{
    int n;
    scanf("%d", &n); //数据很大, 需要用LL读入
    while (n--)
    {
        int a, b, p;
        scanf("%d%d%d", &a, &b, &p); //读入a b
p

        printf("%d\n", qmi(a, b, p)); //计算快速
幂
    }
    return 0;
}

```

Python

AcWing 875

NQ160 AcWing 868 AcWing 868

AcWing 868

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
// Andy 2021.06.04
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100000007;

int primes[N], cnt;
bool st[N];
void get_primes(int n) // 线性筛质数
{
    for (int i = 2; i <= n; i++)
    {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

int main()
{
    int n; cin >> n;
    get_primes(n);
    cout << cnt << endl;
    return 0;
}
```

Python

AcWing 868

NQ161 AcWing 104 AcWing 104

AcWing 104

输入

 见原题

输出

 见原题

范围

 见原题

输入

见原题

输出

见原题

思路

 见AcWing原题描述

技巧

 参考模板库 CONTEXT.md

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n;
int a[N];

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(a, a+n);
    int res = 0;
    //中位数a[n/2],求距离之和的最小值
    for (int i = 0; i < n; i++) res += abs(a[i] - a[n/2]);
    cout << res << endl;
    return 0;
}
```

Python

AcWing 104